DDD × SPRING FRAMEWORK

レイヤードアーキテクチャで実践するドメイン駆動設計

構成要素の役割と、Spring アノテーションの対応づけ

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

Introduction

ドメイン駆動設計（DDD）とは

業務の知識（ドメイン）をソフトウェアの中心に置き、**業務の言葉で書かれたモデル**としてコードに表現し続ける設計手法。

01

ドメインモデル

業務ルールを、データの入れ物ではなく、振る舞いを持つオブジェクトとして表す。

02

ユビキタス言語

業務担当者と開発者が同じ言葉を使い、その言葉をそのままクラス名・メソッド名にする。

03

境界

境界づけられたコンテキストや集約で、モデルの意味と整合性の範囲を区切る。

本資料では、このうち実装に直結する「戦術的設計」の構成要素と、レイヤードアーキテクチャへの配置を扱います。

Introduction

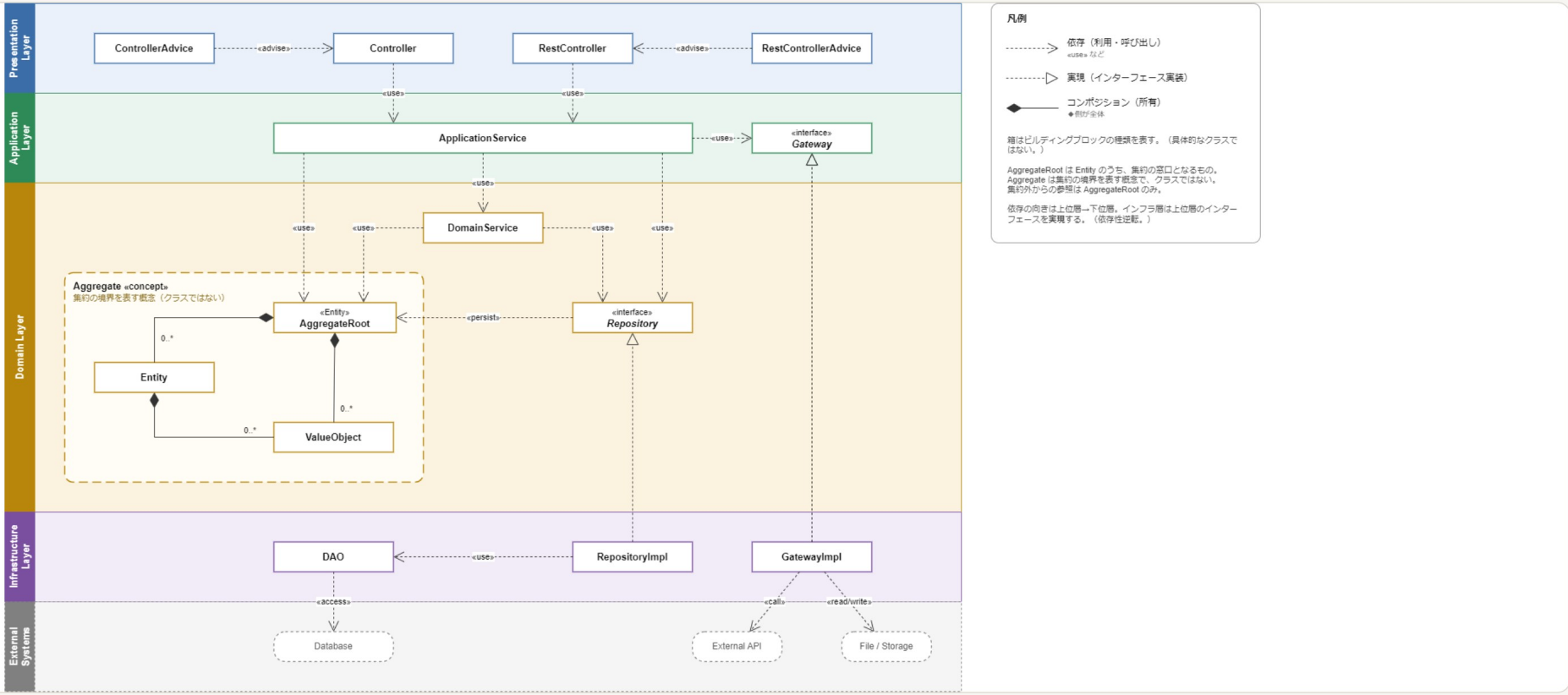
4 つの層と依存の向き

層	責務	主な構成要素
Presentation	入出力を受け付け、HTTP とユースケースの間で変換する	Controller / RestController / (Rest)ControllerAdvice
Application	ユースケースの手順を組み立て、トランザクションと認可の境界になる	ApplicationService / Gateway（interface）
Domain	業務ルールと不変条件を表現する。技術に依存しない	AggregateRoot / Entity / ValueObject / DomainService / Repository（interface）
Infrastructure	DB・外部 API・ファイルなどの技術詳細を実装する	RepositoryImpl / GatewayImpl / DAO

依存は上の層から下の層へ。インフラ層は上位層のインターフェースを**実現**する（依存性逆転）ため、ドメイン層はどの層にも依存しない。

Introduction

構成要素の全体図



Introduction

アノテーション区分の読み方

必須 付けないと動作しない、または規約として必ず付ける

選択 要件や方針に応じて付ける

不要 付けない（付けるべきでない）

共通のルール

- ドメイン層には Spring などフレームワーク固有のアノテーションを持ち込まない（Bean Validation は中立な標準仕様のため可）
- 認可はユースケース単位で、ApplicationService に寄せる
- @PreAuthorize には @EnableMethodSecurity の設定が必要
- @Transactional などはプロキシで動作するため、同一クラス内の呼び出しでは効かない
- jMolecules は、役割の明示と ArchUnit による規約検証のためのライブラリ

Presentation Layer

Controller

画面（HTML）を返す Web アプリケーションの入口

役割

- リクエストを受け取り、入力の形式を検証する
- ApplicationService を呼び出す
- 結果をモデルに詰め、ビュー名を返す

置かないもの

- 業務ルールの判定
- トランザクションの制御

アノテーション	区分	理由・用途
@Controller	必須	Bean として登録され、ハンドラとして検出される
@GetMapping 等	必須	URL と HTTP メソッドをハンドラメソッドに対応づける（@RequestMapping 系）
@Valid / @Validated	選択	フォーム入力を Bean Validation で検証する場合
@PreAuthorize	選択	画面単位で認可したい場合。原則は ApplicationService に寄せる

Presentation Layer

RestController

JSON などのデータを返す API の入口

役割

- リソースの表現を受け取り、返す
- ApplicationService を呼び出す
- 表現はドメインモデルそのもの。直列化の調整は Mix-in でドメインの外に置く

置かないもの

- 外部要求に合わせた、ドメインと無関係な API 独自の形
- 業務ルールの判定

アノテーション	区分	理由・用途
@RestController	必須	@Controller ＋ @ResponseBody。戻り値をボディとして直列化する
@GetMapping 等	必須	エンドポイントの URL と HTTP メソッドを定義する
@RequestBody 等	必須	引数の取得元を指定する。ボディはレスポンスと同じ表現で受け、自動生成項目は Mix-in で読み取り専用にする
@Valid	選択	ドメインモデルに宣言した制約を、受け取り時に検証する
@Validated	選択	検証グループで新規・更新の制約を切り替える場合。クラスに付けると引数の制約も検証する（Spring 6.1 以降は標準で検証されるため、クラスからは除去を推奨）
@ResponseStatus	選択	201 Created など既定以外のステータスを返す場合

Presentation Layer

ControllerAdvice

画面系 Controller の横断処理（例外ハンドリングなど）

役割

- 例外の種類に応じて、エラー画面へ振り分ける
- 全画面に共通するモデル属性・バインド設定を定義する

置かないもの

- 再試行・補償などの業務的な回復処理
- API 向けのエラー応答（RestControllerAdvice へ）

アノテーション	区分	理由・用途
@ControllerAdvice	必須	全 Controller（または指定範囲）に適用される Bean として登録する
@ExceptionHandler	必須	例外の型ごとにハンドラを定義する。この Advice の主目的
basePackages（属性）	選択	RestControllerAdvice と併用するなら実質必須。適用範囲をパッケージで分ける
@InitBinder / @ModelAttribute	選択	全画面共通のバインド設定・モデル属性を定義する場合
@Order	選択	複数の Advice の優先順位を決める場合

Presentation Layer

RestControllerAdvice

API 系 Controller の横断処理（エラー応答の統一）

役割

- 例外を統一形式のエラーレスポンス（ProblemDetail など）に変換する
- 例外に応じてステータスコードを決める
- Spring MVC 標準の例外は ResponseEntityExceptionHandler の継承でまとめて扱える

置かないもの

- 例外を握りつぶす処理

アノテーション	区分	理由・用途
@RestControllerAdvice	必須	@ControllerAdvice ＋ @ResponseBody。戻り値をレスポンスボディとして返す
@ExceptionHandler	必須	例外の型ごとにエラー応答を定義する
@ResponseStatus	選択	ハンドラの応答ステータスを固定する場合。ResponseEntity で返すなら不要
basePackages（属性）	選択	画面系の ControllerAdvice と適用範囲を分ける場合

Application Layer

ApplicationService

1 ユースケース ＝ 1 メソッド。処理の手順を組み立てる

役割

- Repository で集約を取得・保存する
- 判断はドメインオブジェクト・DomainService に委ねる
- Gateway で外部とやり取りする
- トランザクションと認可の境界になる

置かないもの

- 業務ルールの判定そのもの
- SQL・HTTP などの技術詳細

アノテーション	区分	理由・用途
@Service	必須	Bean として登録し、アプリケーション層のサービスであることを明示する
@Transactional	必須	ユースケース ＝ トランザクション境界。更新系は必須、参照系は readOnly = true
@PreAuthorize	選択	ユースケース単位で認可する場合。@EnableMethodSecurity が必要
@Validated	選択	引数（コマンド）をメソッド単位で検証する場合
@TransactionalEventListener	選択	コミット後にドメインイベントを処理する場合（通知・連携など）

Application Layer

Gateway «interface»

Repository に当てはまらない、外部システムとの窓口（抽象）

役割

- 外部 API・ファイル・メールなどとのやり取りを、業務の言葉のメソッドで定義する
- 引数・戻り値はアプリケーション／ドメインの型にする

置かないもの

- 外部 API の DTO・ライブラリの型
- 技術名の命名（XxxApiClient など）

アノテーション	区分	理由・用途
（なし）	不要	純粋なインターフェース。Spring に依存させない

```
public interface PaymentGateway {
    PaymentResult charge(OrderId orderId, Money amount);
}
```

Repository との使い分け

集約の保存・取得なら Repository、それ以外の外部とのやり取りは Gateway。技術ではなく意味で決める。

Domain Layer

Aggregate «concept»

一緒に整合性を保つオブジェクトのまとまり。概念であり、クラスではない

性質

- Root と境界を 1 つずつ持つ
- 1 トランザクションで更新するのは 1 集約
- 他の集約は ID で参照する

コード上の現れ方

- 1 集約 ＝ 1 パッケージ
- Repository は集約ごとに 1 つ
- Root の型が集約を代表する

アノテーション	区分	理由・用途
（なし）	不要	概念のため、対応するクラス・アノテーションはない

```
domain/order/              // Order 集約
├─ Order.java             // AggregateRoot
├─ OrderId.java
├─ OrderDetail.java       // ValueObject
└─ OrderRepository.java   // interface
```

境界の強制

集約内部のクラスやコンストラクタはパッケージプライベートで隠し、ArchUnit などのアーキテクチャテストで補う。

Domain Layer

AggregateRoot «Entity»

Entity のうち、集約の唯一の窓口となるもの

役割

- 集約内の不変条件を守る更新メソッドを提供する
- 集約内の Entity・VO の変更は必ず Root 経由
- 楽観ロックのバージョンを持つ

置かないもの

- 他の集約のオブジェクト参照（ID で持つ）
- setter による外部からの直接変更

アノテーション	区分	理由・用途
@Component 等	不要	ドメインオブジェクトは Bean にしない。コンストラクタやファクトリで生成する
@AggregateRoot（jMolecules）	選択	役割を明示し、規約を検証する場合。マーカーインターフェースでも代替できる
@Entity / @Id / @Version（JPA）	選択	ドメインモデルを JPA エンティティと兼用する場合のみ。分離を推奨
@DomainEvents（Spring Data）	選択	保存時にドメインイベントを発行する場合。ドメイン層が Spring Data に依存する点に注意

Domain Layer

Entity

同一性（ID）で区別されるオブジェクト。ここでは集約の内側にあるもの

役割

- 集約内で一意なローカル ID を持つ
- 自身の状態を変える振る舞いを持つ（呼び出しは Root 経由）

置かないもの

- 専用の Repository（独立して更新したいなら別集約にする）
- 集約の外からの直接変更

アノテーション	区分	理由・用途
@Component 等	不要	ドメインオブジェクトは Bean にしない
@NotNull 等（Bean Validation）	選択	形式的な制約を宣言し、コンストラクタの最後で Validator により自己検証する
@Entity（jMolecules）	選択	役割を明示し、規約を検証する場合。JPA の @Entity と同名のため import に注意
@Entity（JPA）	選択	ドメインモデルを JPA と兼用し、別テーブルに保存する場合のみ

Domain Layer

ValueObject

値そのもので等価性が決まる、不変のオブジェクト

役割

- 生成時に値の妥当性を検証し、不正な値を作らせない
- 値に関する振る舞い（計算・比較・整形）を持つ
- Java の record で実装する

置かないもの

- setter・可変の状態
- ID（ID が必要なら Entity）

アノテーション	区分	理由・用途
@Component 等	不要	ドメインオブジェクトは Bean にしない
@NotNull 等（Bean Validation）	選択	形式的な制約を宣言し、標準形のコンストラクタの最後で Validator により自己検証する
@ValueObject（jMolecules）	選択	役割を明示し、規約を検証する場合
@Embeddable（JPA）	選択	JPA と兼用し、親テーブルの列として保存する場合（利用側は @Embedded）

Domain Layer

DomainService

単一の集約に置くと不自然な業務ルール（集約間の相関チェックなど）

役割

- 複数の集約にまたがる判定・計算
- 既定は、ApplicationService から集約を受け取って判定する
- 一意性チェックなど、問い合わせ自体がルールなら Repository を使ってよい（読み取りのみ）

置かないもの

- 保存（save）とトランザクション制御
- Gateway の呼び出し（上位層のため参照不可）

アノテーション	区分	理由・用途
@Service / @Component	選択	DI で注入する場合。ドメイン層の Spring 依存を避けるなら @Configuration の @Bean で登録する
@Transactional	不要	トランザクション境界は ApplicationService が持つ
@PreAuthorize	不要	認可はユースケース単位（ApplicationService）で行う

Domain Layer

Repository «interface»

集約を永続化・再構成するための抽象。集約ごとに 1 つ

役割

- Root の型で集約を出し入れする
- 検索条件を業務の言葉で表す

置かないもの

- JpaRepository など技術インターフェースの継承
- 集約内の Entity 単位のメソッド

アノテーション	区分	理由・用途
（なし）	不要	純粋なインターフェース。Spring・JPA に依存させない
@Repository	不要	インターフェースではなく、実装クラス（RepositoryImpl）に付ける

```
public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    void save(Order order);
}
```

Infrastructure Layer

RepositoryImpl

Repository インターフェースの実装。集約と永続化モデルを相互に変換する

役割

- DAO を使って読み書きする
- 集約 ⇄ テーブル行・永続化エンティティを変換する
- 集約単位でまとめて保存する

置かないもの

- 業務ルール
- トランザクションの開始（ApplicationService から伝播させる）

アノテーション	区分	理由・用途
@Repository	必須	Bean として登録し、永続化技術の例外を DataAccessException に変換する
@Transactional	選択	propagation = MANDATORY で、トランザクション外からの呼び出しを検出する場合

Infrastructure Layer

GatewayImpl

Gateway インターフェースの実装。外部 API・ファイルなどの技術詳細を閉じ込める

役割

- HTTP クライアント・ファイル I/O を呼び出す
- 外部の DTO ⇄ アプリケーションの型を変換する（腐敗防止）
- 外部のエラーをアプリケーションの例外に変換する

置かないもの

- 業務ルール
- 外部の型を戻り値で漏らすこと

アノテーション	区分	理由・用途
@Component	必須	Bean として登録する。永続化ではないため @Repository ではなく @Component
@Retryable	選択	一時的な障害を再試行する場合（Spring Framework 7 以降は標準、以前は Spring Retry）
@CircuitBreaker	選択	外部の障害が連鎖しないよう遮断する場合（Resilience4j）
@Cacheable	選択	変化の少ない参照データをキャッシュする場合
@HttpExchange 等	選択	宣言的 HTTP クライアント（HTTP Interface）を実装の中で使う場合

Infrastructure Layer

DAO

永続化技術に直結したデータアクセス（テーブル・SQL 単位）

役割

- SQL・クエリを実行する
- テーブル構造に対応したデータを読み書きする

置かないもの

- 集約の組み立て（RepositoryImpl の役割）
- アプリケーション層からの直接呼び出し

アノテーション	区分	理由・用途
@Mapper（MyBatis）	必須	MyBatis の場合。Mapper インターフェースとして検出させる
@Entity / @Table / @Id（JPA）	必須	JPA の場合。DAO 側の永続化エンティティにマッピングを付ける
@Repository	必須	JdbcClient などで自前実装する場合。Spring Data のインターフェースには不要
@Query	選択	Spring Data で JPQL や SQL を明示する場合

Summary

必須アノテーション早見表

Spring のアノテーションは外側の層に集まり、ドメイン層は原則アノテーションなし

構成要素	必須
Controller	@Controller / @GetMapping 等
RestController	@RestController / @GetMapping 等
ControllerAdvice	@ControllerAdvice / @ExceptionHandler
RestControllerAdvice	@RestControllerAdvice / @ExceptionHandler
ApplicationService	@Service / @Transactional
Gateway	なし
Aggregate	なし（概念）

構成要素	必須
AggregateRoot	なし
Entity	なし
ValueObject	なし
DomainService	なし（DI するなら @Service）
Repository	なし
RepositoryImpl	@Repository
GatewayImpl	@Component
DAO	技術次第（@Mapper など）

Appendix

付録A：Bean Validation の適用パターン

パターン	検証のタイミング	利点	注意点
A：プレゼン層で検証	@Valid / @Validated によるリクエスト受け取り時	項目ごとのエラー情報を得やすい。検証グループで新規・更新の制約を切り替えられる	ドメインのコード内で生成したインスタンスは検証されない
B：自己検証	コンストラクタによるインスタンス生成時	どの経路で生成されても、不正なインスタンスが存在しない	違反は ConstraintViolationException。Jackson 経由では包まれて届く。グループの切り替えは難しい

選び方

常に正しい状態を保証したいなら B。エラー応答の作り込みや、新規・更新での制約の切り替えを重視するなら A。B を採る場合、リクエストボディは逆直列化の時点で検証されるため @Valid は不要になる。

Appendix

付録A-1：プレゼン層で検証する例

```
// domain/member/Member.java
public record Member(
        @Null(groups = Create.class)
        MemberId id,
        @NotBlank @Size(max = 50)
        String name,
        @Email
        String email) {
}

// 新規作成時に追加で検証するグループ
public interface Create extends Default {}
```

Create を Default の派生にすると、通常の制約もあわせて検証される。

```
// presentation/member/MemberController.java
@RestController
@RequestMapping("/members")
public class MemberController {
    private final MemberService service;
    // コンストラクタは省略

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Member create(
            @Validated(Create.class)
            @RequestBody Member member) {
        return service.register(member);
    }
}
```

Appendix

付録A-2：エンティティ自身に検証させる例

```
// domain/member/Member.java
public record Member(
        MemberId id,
        @NotBlank @Size(max = 50) String name,
        @Email String email) {

    public Member(MemberId id,
                  String name, String email) {
        this.id = id ≠ null
                ? id : MemberId.generate();
        this.name = name;
        this.email = email;
        DomainValidator.validate(this);
    }
}
```

```
// domain/shared/DomainValidator.java
public final class DomainValidator {
    private static final Validator VALIDATOR =
        Validation.buildDefaultValidatorFactory()
            .getValidator();

    public static <T> void validate(T target) {
        var v = VALIDATOR.validate(target);
        if (v.isEmpty()) return;
        throw new ConstraintViolationException(v);
    }
}
```

検証は標準形のコンストラクタの最後で行う。コンパクトコンストラクタは本体がフィールド代入より前に動くため使えない。

Appendix

付録A-3：自己検証の違反をエラー応答にする

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler
    public ProblemDetail onViolation(
            ConstraintViolationException e) {
        return ProblemDetail.forStatusAndDetail(
        HttpStatus.BAD_REQUEST, e.getMessage());
    }
    @ExceptionHandler
    public ProblemDetail onUnreadable(
            HttpMessageNotReadableException e) {
        var cause = e.getRootCause();
        if (cause instanceof
                ConstraintViolationException c)
            return onViolation(c);
        return ProblemDetail.forStatus(400);
    }
}
```

2 つの経路で届く

- ドメインのコード内での違反は、ConstraintViolationException のまま届く
- リクエストの逆直列化中の違反は、HttpMessageNotReadableException に包まれて届く

対処

包まれた例外は getRootCause() から元の違反を取り出し、どちらも同じ 400 の ProblemDetail に揃える。

Appendix

付録B：Mix-in でプレゼン層のビューを定義する

```
// domain/order/Order.java
public record Order(
        OrderId id,
        CustomerId customerId,
        OrderDetails details,
        long version) {

    public Order {
        if (id == null) id = OrderId.generate();
    }
}
```

ドメインには Jackson のアノテーションを付けない。自動生成項目はコンストラクタで補い、リクエストで省略・無視されても生成できるようにする。

```
// presentation/order/OrderView.java
@JsonMixin(Order.class)
public abstract class OrderView {
    @JsonProperty(access = Access.READ_ONLY)
    abstract OrderId id();

    @JsonIgnore
    abstract long version();
}
```

@JsonMixin（Spring Boot）で自動登録される。Boot を使わない場合は objectMapper.addMixIn(Order.class, OrderView.class) で登録する。version は本文に出さず ETag で扱う。